Markov assumption property:

$$p(x_1, \ldots, x_n) = p(x_n|x_{n-1})\, p(x_{n-1}|x_{n-2}) \ldots p(x_2|x_1)\, p(x_1)$$

```
LSTM → forget gate: sigmoid, long-term mem | output gate: sigmoid, short-term mem
```

Trainable params (input = 10, hidden = 5):

$$\bar{h}_t = \tanh(h_{t-1}W + x_tU + b)$$

Params = $10 \times 5 + 5 \times 5 + 5$

Week8:

Q12 Word2Vec → skip-gram: center predicts context (e.g. 'away'→'the, mouse, runs, from, the, cat') |
CBOW: context predicts center (e.g. 'the, mouse, runs, from, the, cat'→'away')
Q6 Skip-gram drawbacks →1. softmax expensive; 2.predicted probs tiny & hard to distinguish; 3.depends on window size
Q7 SGNS (skip-gram with negative sampling) → output layer uses sigmoid (NOT softmax; outputs do NOT sum to 1); 1.randomly sample negative words; 2.maximize P(tw,cw) for the positive pair; 3.minimize P(tw, neg) for sampled negatives
Q89 Skip-gram obj: J=∏□∏□p(w□+□|w□); t = center (target) word; j∈[−c,c], j≠0; maximize J(θ) |
CBOW obj: J=∏□p(w□|w□-(c)…w□+(c)); t = center word; window size = 2c+1
Q10 Skip-gram → vocab=200, embed=100 → U[200,100], V[100,200]; input=one-hot(5); hidden=h=row5(U)
Q11 CBOW → vocab=100, embed=150 → U[100,150], V[150,100]; input=avg(one-hot10,120,130,140); hidden=h=avg(rows10,20,30,40 of U)

Week9

Q1 Encoder–Decoder →**(1) Encoder:** encodes input → context vector **(2) Decoder:** reads context → generate output sequence **(3) Context vector:** summarizes input sequence
Q2 Seq2seq (MT) →**(1)** Encoder & decoder are **RNNs (2) Context vector** = last hidden state of encoder
Q3 Seq2seq log-likelihood →**(1) c** = summary of source seq x1:□x **(2) q**□-1 = summary of generated seq y1:□-1 **(3) P(y**□**|q**□-1**,c,θ)** → dense layer predict next word y□
Q4 Seq2seq conditional P(y□**|q**□-1**,c,θ)** → **(1) Diagram (a):** basic encoder–decoder (uses c) **(2) Diagram (b):** same with explicit c input to each decoder step **(3) Diagram (c):** concatenates h□x and c → also valid
Q5 Seq2seq decoding →**(1) Phase 1**: encode x1:□x → context c = h_{Tx} **(2) Phase 2**: feed BOS to decoder; init decoder state with encoder's last hidden h_{Tx}
(3) Greedy strategy: at each step choose argmax P(y_t | q_{t−1}, c, θ)
Q6 Timely-varying context advantages → (1) Better for long sequences; adapts dynamically to inputs (2) Focuses on important words when generating each output
(vs fixed-length: static, less capable for long-range info)
Q7 Global attention → time-varying context uses all encoder hidden states; computed as a (weighted) linear combination of all encoder states
Q8 Local attention → time-varying context uses encoder hidden states within a selective window; formed as a (weighted) linear combination over that window
Q10 Positional Encoding (Transformer) → encodes token position within a sentence; added to token embeddings
Q11 LayerNorm (Transformer) → normalize over feature dim (d_model), not batch; learnable γ,β; better than BatchNorm for sequential data
Q12 Self-Attention → (1) z_i depends on its own x_i and all other x□ (j≠i) (2) More similar x□ to x_i → higher attention weight → larger contribution to z_i
Q13 Self-Attention →**(A)** use W_Q, W_K, W_V to compute Q,K,V **(C)** compute attention scores B = QKᵀ ∈ ℝ[L,L] **(D)** Q,K are two linear projections (views) of X
(E) apply softmax to B → attention probs A ∈ ℝ[L,L] **(G)** output Z = A V (weighted sum of values)
Q14 Multi-head Self-Attention →**(A)** each head has its own W_Q, W_K, W_V **(C)** heads run independently (in parallel) **(F)** concat(all heads) → linear W_O → final output
Q15 Cross-Attention →(A) **inject encode**r output into decoder layers (C) **decoder input** → Q; **encoder output** → K,V (F) used in computing the decoder output

Week 10

Q1 Self-Attention → (A) compute all token **embeddings** in **parallel** (C) Q,K,V **derived** from **same source** sequence X (E) captures **intra-sequence** dependencies within a single sequence
Q2 Cross-Attention → (A) compute token **embeddings** in **parallel** (D) **Q** from **target**; **K,V** from **source** (F) captures inter-sequence dependencies (source ↔ target)
Q3 CNN drawbacks → **limited** global context; **weak** at modeling relative spatial relationships; **locality-biased** (locality sensitivity)
Q4 ViT → **(1) token = image patch**; **(2)**flatten patch → linear projection → token embedding; **(3)**add learnable [CLS] token; **(4)**final MLP reads [CLS] for prediction
Q5 ViT global info → Multi-head self-attention enables global token interactions + final [CLS] token aggregates all tokens for prediction
Q6 ViT → capture global image info; model long-term patch dependencies; need massive datasets to train
Q7 Swin Transformer → smaller patches [3,4,4]; linear proj → tokens shaped [C, H/4, W/4]; patch merging: **downsample** (H,W)/2 while **doubling channels**
Q8 Swin Patch Merging → merge 2×2 neighbors, concat → linear; input [C, H/4, W/4] → output [2C, H/8, W/8]
Q9 Swin Window Self-Attention → divide tokens into local windows& apply SA per window; output shape = input shape
Q10 Swin Window SA → speeds up standard SA; limits interactions to tokens within the same local window
Q11 Shifted Window SA → (B) token can interact with tokens in different windows (C) enables cross-window interaction (D) shifts windows (right/down) to form new windows (F) output shape = input shape
Q11 ViT fine-tuning additional components → add modules that respect the original ViT computation; train/optimize the added components (backbone largely unchanged)
Q12 ViT fine-tuning additional components → add modules that respect the original ViT computation; train/optimize the added components
Q13 Prompt-tuning for ViTs → (A) Add learnable prompts to **token embeddings**, fine-tune prompts only
Q14 Adapter-tuning for ViTs → (B) Add adapters to **pointwise networks**, fine-tune adapters only
Q15 LoRA-tuning for ViTs → (C) Add **low-rank matrices** to Q, K, V → fine-tune low-rank parts only

Week 11

Q1 Generative model → Generate **realistic examples from noise** that mimic the data distribution
Q2 GANs → (B) Discriminator compares **real vs. generated distributions** (C) Noise → Generator → Fake samples (D) Discriminator **distinguishes real/fake** examples
Q3 GAN tasks → (A) Discriminator **classifies real vs. fake data** (B) Discriminator outputs **high for real, low for fake** (D) Generator **fools discriminator** by making data **mimic real**
Q4 GAN analogy → (C) Generator = **fake money machine** (D) Discriminator = **police detecting fakes**
Q5 GANs

$$\min_G \max_D J(G,D) = \mathbb{E}_{x\sim p_d(x)}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z))]$$

$$\min_G \max_D J(G,D) = \mathbb{E}_{x\sim p_d(x)}[\log(1 - D(x))] + \mathbb{E}_{z\sim p(z)}[\log D(G(z)]$$

Q6 update the generator and discriminator alternatively

$$\max_D J(G,D) = \mathbb{E}_{x\sim p_d(x)}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z))]$$

$$\max_G \mathbb{E}_z[\log(D(G(z))]$$

$$\max_D J(G,D) = \mathbb{E}_{x\sim p_d(x)}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z))]$$

$$\min \mathbb{E}_z[\log(1 - D(G(z))]$$

Q7 pg overlaps pd ↑ → (B) harder; (C) acc → ~50% (random guess)
Q8 correct about discriminator? (1)sigmoid output layer (2) $D(x)$ **real** (3) **1 −** $D(x)$ **fake**

Q9 correct for optimal D^*? → $D^*(x) = p_d/(p_d + p_g) = 1/(1 + p_g/p_d)$

Q10 Nash equilibrium of GAN → *D(x)=0.5, p_g=p_d**

Q11GANs **Mode collapse** :generated data can cover only a few modes in real data and miss many other modes
Q12 Mode collapsing → (C) cannot solve **min–max** problem perfectly; (D) **generator** update has no constraints to **cover all modes**
Q13 GANs training issues: Mode collapse, unrealistic generated images for complex datasets, unstable training
Q14 ConvTranspose2d

$$H_{out} = (H_{in} - 1) \times stride - 2 \times padding + kernel_size + output_padding$$

Q15 DMs — Forward: shrink + add noise to clean images → noisy versions. **Backward:** go backward from noise, denoising at each step.

Q16 DMs Forward : $x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon$ where $\epsilon \sim N(0, \mathbb{I})$

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \text{ where } \epsilon \sim N(0,\mathbb{I}) \text{ and } \bar{\alpha}_t = \prod_{i=1}^{t}(1-\beta_i)$$

Q17 — U-Net εθ(x□,t): (A) minimize ‖εθ(x□,t) − ε‖² with x□ = √ᾱ□·x₀ + √(1−ᾱ□)·ε, ε ~ N(0,I); (C) predict the Gaussian noise ε in x□; (D) denoise noisy images → clean image

week1

Q10 CE loss range → [0, +∞) (min 0 when py=1p_y=1py=1; unbounded as py→0p_y\to0py→0).
Q12 match → **KL(p||q)** → (2) $\sum p_m \log \frac{p_m}{q_m}$; **Cross-entropy** $H(p,q)$ → (3) $-\sum p_m \log q_m$; **Entropy** $H(p)$ → (1) $\sum p_m \log \frac{1}{p_m}$.

week2

Q12 → 5th row of h^3 = logits of the **5th sample** in the mini-batch (not "5th logit across all samples"). **C** ✅
Q14 4th row of **p** → prediction probabilities of the **4th sample** in the mini-batch.
Q15 architecture → **C**: 16→10(ReLU)→20(ReLU)→15(ReLU)→26 **(no output activation)**.
Q16 total number parameters 16x10 +10 + 10x20 + 20 + 20x15 +15 + 15x26 +26

week3

Q9 [4,10] after softmax → row = sample, col = class prob → 2nd row = prediction probabilities (10-dim) for the 2nd image in the batch.
Q10 Conv2D purpose → (B) process 2D/3D tensors; (C) learn filters to detect patterns/features inside an input tensor
Q11 pooling layers? →preserve crucial info (make reps more robust). downsample: reduce input size/params/overfitting -manageable.
Q12 BatchNorm? → mitigates **overfitting** (regularization effect).**train**: use batch mean/std. **test**: use population (running) mean/std
Q1 correct: **A, C, E** — minimize $J(\theta) = \Omega(\theta) + \frac{1}{N}\sum CE(y_i, f(x_i;\theta))$ with $\Omega(\theta) = \lambda \sum_k \|W^k\|_F^2$: (A) drives more $W^k_{i,j} \to 0$; (C) encourages **simple models**; (E) **combats overfitting**.
Q2 **CE= empirical loss**; minimizing it makes the model **fit the training set**; minimizing it **alone** (without Ω(θ)) can lead to **overfitting**.
Q6 (SGD): $\theta_{t+1} = \theta_t - \frac{\eta}{b}\sum_{k=1}^{b} \nabla_\theta \ell(x_{i_k}, y_{i_k}; \theta_t)$ [C]; Q7 (GD): $\theta_{t+1} = \theta_t - \frac{\eta}{N}\sum_{i=1}^{N} \nabla_\theta \ell(x_i, y_i; \theta_t)$ [A].
Q10 (PyTorch) → **B, C, D, E**: ∂ℓ/∂W, ∂ℓ/∂b; W.grad/W.data; b.grad/b.data (x requires_grad=False ⇒ A,F false).
Q11 → C, E: l.backward() does **backprop** to compute W.grad & b.grad, traversing **from loss l back through ŷ to inputs/params** (not forward; not .data).
Q13 → D: **Jacobian** for element-wise activation is **∂h/∂h̃ = diag(σ′(h̃))** (e.g., sigmoid: diag(σ(h̃)(1−σ(h̃))); ReLU: diag(1_{h̃>0})).

week5

Q23 Xavier **initialization** -> Sigmoid/Tanh || He initialization -> ReLU
Q45 gradient vanishing->small gradient at any layer || **exploding** big at any layer
Q6 layer reduce gradient vanishing -> Batch normalization/Skip connection/ReLU **Q7 Underfitting** → (simple model for complex data) & (both training/valid accuracies low) **Overfitting** → (powerful deep nets on simple data) & (training high, valid low)
Q13 Reduce overfitting →Early stopping, Adding more data, Weight regularization, Using Dropout

week6

Q1 → (deep learning: training signal from classifier can improve feature extractor) ; (traditional: training signal cannot improve feature extractor).
Q4 receptive field → C higher-layer receptive fields become larger; D neuron value is computationally relevant to its receptive field.
Q5 residual block → A: Conv → BN → ReLU → Conv → BN + skip add → ReLU
Q9 ResNet → 1×1 Conv in skip-connection to change #channels/shape; ResNet = many blocks → many residual blocks, each with several conv + activations.

Q10: x_adv and x look very similar under human perspective; and $\arg\max_m f_m(x_{\text{adv}}) \neq \arg\max_m f_m(x)$.
Q11: The constraint $x_{\text{adv}} \in B_\epsilon(x) = \{x' : \|x' - x\|_\infty \le \epsilon\}$ makes x_adv and x look similar to humans; the max per-pixel absolute difference is $\le \epsilon$.

$$x_{\text{adv}} = \arg\max_{x' \in B_\epsilon(x)} \ell(f(x';\theta), y)$$

Q12 (untargeted): maximally decrease the chance of label y; maximally increase the chance of some label y'≠y; it is an untargeted attack.

$$x_{\text{adv}} = \arg\min_{x' \in B_\epsilon(x)} \ell(f(x';\theta), y_{\neq}), \quad y_{\neq} \neq y.$$

Q13 (targeted): maximally increase the chance of a specific target label y≠y; it is a targeted attack.**Q14 (adversarial training)**: augment data each iteration with an adversarial attack (e.g., PGD); update the model on clean and adversarial images to their ground-truth labels; final loss = loss over clean + adversarial examples.

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$; $\sigma'(z) = \sigma(z)(1-\sigma(z))$; range $(0,1)$; +probability-like; −non-zero-centered & vanishing gradients for large $|z|$.
- **tanh**: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$; $\tanh'(z) = 1 - \tanh^2(z)$; range $(-1,1)$; +zero-centered; −saturates → vanishing gradients.
- **ReLU**: $\mathrm{ReLU}(z) = \max(0,z)$; $\mathrm{ReLU}'(z) = \mathbf{1}[z>0]$; range $[0,\infty)$; +fast, sparse, mitigates vanishing; −"dying ReLU".
- **Leaky ReLU (alt.)**: $\mathrm{LReLU}(z) = \max(\alpha z, z)$ with $\alpha \approx 10^{-2}$; derivative α (for $z \le 0$) / 1 (for $z > 0$) +alleviates dying ReLU.
- **Softmax (multiclass)**: $s_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$; Jacobian $\nabla s = \mathrm{diag}(s) - ss^\top$; typically used with cross-entropy.

0) Notation (skinny header)

- **one-hot**: $(\mathbf{1}_y)_c = \delta_{c,y}$
- **softmax**: $p_c = \frac{e^{h_c}}{\sum_j e^{h_j}}$; **CE**: $\ell = -\log p_y$
- **key gradient**: $\boxed{\frac{\partial \ell}{\partial \text{logits}} = p - \mathbf{1}_y}$

2) CNN: conv/pool shapes & spatial-relation caveat

Shape (conv/pool). $H_{out} = \left\lfloor \frac{H+2P-K}{S} \right\rfloor + 1$ (same for W). Channels = #filters.
Caveat. Local receptive fields + pooling ⇒ strong "is there a part" detectors, weak **relative geometry** (mis-arranged parts can still fire).
One-liner. "CNN composes local detectors; without explicit relational/global modeling, wrong part layout can still be classified as the object."

1) Overfitting / Regularization / Early-Stopping / BN / Aug / Label Smoothing

Idea. Overfitting = great train, poor test. Fix with capacity control + early feedback.
Objective. $J(\theta) = \Omega(\theta) + \frac{1}{N}\sum_i \ell(y_i, f(x_i;\theta))$
Formulas.

- L2: $\Omega = \lambda \sum_k \|W_k\|_F^2$.
- BN: $\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, $z_{BN} = \gamma \hat{z} + \beta$
- Label smoothing (K classes): $\tilde{\mathbf{1}}_y = (1-\alpha)\mathbf{1}_y + \frac{\alpha}{K}\mathbf{1}$

One-liners.

- *Overfitting?* "Low train error, high test error; use early stopping, L2/L1, dropout, BN, data augmentation, label smoothing."
- *Early stop criterion.* "Stop when train loss ↓ but valid loss ↑ (first turning point)."

3) Adversarial examples & adversarial training

Threat set. $B_\epsilon(x) = \{x' : \|x' - x\| \le \epsilon\}$ (often ℓ_∞).
FGSM. Untargeted: $x' = x + \epsilon\,\mathrm{sign}(\nabla_x \ell(f(x), y))$; Targeted to class t: $x' = x - \epsilon\,\mathrm{sign}(\nabla_x \ell(f(x), t))$.
PGD. Iterative FGSM + projection $\Pi_{B_\epsilon(x)}$.
Adv-train. $\min_\theta \mathbb{E}_{(x,y)}\left[\max_{\|\delta\|\le\epsilon} \ell(f(x+\delta;\theta), y)\right]$.

One-liner. "Adv-training ≈ data expansion but uses **worst-case, gradient-guided** perturbations (not random geometric aug)."

4) RNN / LSTM / GRU (sequences & long dependencies)

Why vanish/explode? BPTT multiplies Jacobians across time → exponential decay/blow-up.
Vanilla RNN. $h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$, $\hat{y}_t = \mathrm{softmax}(W_{hy}h_t + b_y)$.
LSTM.

$$i_t = \sigma(x_t U_i + h_{t-1}W_i), \quad f_t = \sigma(\cdot), \quad g_t = \tanh(\cdot),$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad o_t = \sigma(\cdot), \quad h_t = o_t \odot \tanh(c_t)$$

GRU. $z_t = \sigma(x_t U_z + h_{t-1}W_z)$, $r_t = \sigma(\cdot)$
$\tilde{h}_t = \tanh(x_t U + (r_t \odot h_{t-1})W)$, $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

One-liners.

- "LSTM's cell c_t creates a near-identity pathway for gradients—solves long-term deps better."
- "GRU merges states, fewer params—faster on small/real-time; LSTM steadier on very long deps."
- "Sequence **classification** uses h_T; **tagging** uses per-step $\hat{y}_t$."

5) Word2Vec (beefed up)—Skip-gram / CBOW / Negative Sampling / Subsampling

Goal. Learn word vectors where **similar contexts → nearby embeddings.**
Two pretext tasks. Skip-gram (center → context); **CBOW** (context → center). SG handles rare words better; CBOW is faster/robust.
Embeds. $W_{in}, W_{out} \in \mathbb{R}^{V\times d}$.

SGNS loss (positive pair (w,c) + K negatives n_k).

$$\boxed{\mathcal{L}_{\text{SGNS}} = -\log\sigma(u_c^\top v_w) - \sum_{k=1}^{K}\log\sigma(-u_{n_k}^\top v_w)}$$

$v_w = W_{in}[w,:]$, $u_c = W_{out}[c,:]$

Sampling tricks.

- **Neg distribution:** $P_{neg}(w) \propto f(w)^{0.75}$ (balances head/tail).
- **Subsample frequent words:** $P_{drop}(w) = 1 - \sqrt{\frac{t}{f(w)}}$ (e.g. $t \approx 10^{-5}$).

One-liners.

- "Negative sampling reduces $O(V)$ softmax to $O(K)$ binary tasks—fast."
- "SG vs CBOW: SG better on rare words; CBOW faster & stable."
- "0.75 power: trade-off—prevent head words from dominating without over-punishing them."
- "Linear analogies (king−man+woman≈queen) emerge from distributional constraints."

Side-bar: FNN Softmax-CE Backprop (paste-ready for calc questions)

Forward. $\bar{h}^1 = xW^1 + b^1$, $h^1 = \mathrm{ReLU}(\bar{h}^1)$, $\bar{h}^2 = h^1W^2 + b^2$, $p = \mathrm{softmax}(\bar{h}^2)$, $\ell = -\log p_y$.
Gradients. $\boxed{\partial\ell/\partial\bar{h}^2 = p - \mathbf{1}_y}$; $\partial\ell/\partial W^2 = (h^1)^\top(p - \mathbf{1}_y)$; $\partial\ell/\partial b^2 = p - \mathbf{1}_y$; $\partial\ell/\partial h^1 = (p - \mathbf{1}_y)(W^2)^\top$; $\mathrm{ReLU}' = \mathbf{1}[\bar{h}^1 > 0]$; $\partial\ell/\partial W^1 = x^\top(\partial\ell/\partial\bar{h}^1)$.

6) Seq2Seq & Attention (includes the Q21 skeleton)

BOS/EOS. BOS seeds decoding; EOS stops (variable-length outputs).
MLE + chain rule. $\log P(y|x) = \sum_{t=1}^{T_y}\log P(y_t|y_{<t}, x;\theta)$.
Fixed c bottleneck. Using a single vector $c = h_{T_x}$ to summarize long inputs loses info.
Global attention.

$$a_t(s) = \mathrm{softmax}_s(\mathrm{score}(q_t, h_s)), \qquad \boxed{c_t = \textstyle\sum_s a_t(s)\,h_s}$$

(score = dot / general / concat)

One-liners.

- "Chain rule turns sentence log-likelihood into a sum of local log-probs."
- "Attention makes c_t time-varying and aligns decoder steps to the right source tokens."

7) Transformer / ViT (beefed up)—Attention, PE, shapes, complexity

Encoder block. MHA → residual + LayerNorm → FFN $(d \to 4d \to d)$ → residual + LayerNorm.
Scaled dot-product attention.

$$\boxed{\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V}$$

Multi-Head: $\mathrm{Concat}(\mathrm{head}_i)W^O$, $\mathrm{head}_i = \mathrm{Attn}(QW_i^Q, KW_i^K, VW_i^V)$.
Why scale by $1/\sqrt{d_k}$. Prevent softmax saturation when d_k is large → stabler gradients.
Positional encoding. Sinusoidal:

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right),\ \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

(or learnable).
Masks. Causal mask (upper-triangular $-\infty$) for autoregressive; cross-attention uses Q from decoder, K, V from encoder.
ViT patchify. Image $(H, W, 3)$ → patches $P \times P$; **#tokens** $N = \frac{HW}{P^2}$. Each patch flattened → linear $W_E \in \mathbb{R}^{(P^2\cdot 3)\times d}$; sequence is $[\mathrm{CLS}, x_1W_E, \ldots, x_NW_E] + \mathrm{PE}$.
Complexity. Self-attn $O(N^2 d)$ with N tokens; larger $P \Rightarrow$ smaller $N \Rightarrow$ cheaper but less detail.

One-liners.

- "Multi-head captures diverse relations/scales in parallel—stronger & stabler."
- "Without positional info, self-attn is permutation-invariant—order is lost."
- "Encoder-only = bidirectional understanding; Decoder-only = autoregressive generation; Encoder-Decoder = translation (with cross-attn)."
- "ViT [CLS] is a learnable global aggregator feeding the classifier."
- "$N = \frac{HW}{P^2}$: bigger P → fewer tokens → cheaper attention (but coarser)."

8) GAN (beefed up)—objective, updates, equilibrium, DCGAN, eval, issues

1-liner concept. G generates fakes from noise $z \sim p(z)$; D discriminates real vs fake.
Objective (memorize).

$$\boxed{\min_G \max_D\ \mathbb{E}_{x\sim p_d}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]}$$

Two-step training.

- **Step-D:** ascend $\mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$
- **Step-G:** descend $\mathbb{E}[\log(1 - D(G(z)))]$ or ascend $\mathbb{E}[\log D(G(z))]$ (non-saturating, stabler)
  Optimal D. $D^*(x) = \frac{p_d(x)}{p_d(x) + p_g(x)}$; at equilibrium $p_g = p_d \Rightarrow D = 0.5$.
  DCGAN shape (28×28 quick mental model).
- G: noise → Linear/Unflatten → ConvT ×2 → `[B,1,28,28]` (ReLU/BN; last Tanh)
- D: `[B,1,28,28]` → Conv/stride → Conv/stride → Flatten → Linear+Sigmoid → `[B,1]`
  Eval (IS intuition). High per-image confidence (low entropy) **and** diverse labels overall (high entropy) ⇒ high IS.
  Issues. Mode collapse; minimax instability/gradient saturation.

One-liners.

- "At optimum $p_g = p_d$: $D(x) = 0.5$ everywhere—indistinguishable."
- "Use the non-saturating generator loss for more stable gradients."

----------one lines answers----------

1) Overfitting / Regularization / Early-Stopping / BN / Aug / Label Smoothing

- **Overfitting (def + fix):** "Overfitting = low train error but high test error; fix with early stopping, L2/L1, dropout, BN, data augmentation, label smoothing, or more data."
- **Underfitting (contrast):** "Underfitting = model class too simple (both train & test errors high); increase capacity, train longer, reduce regularization."
- **Early stopping (how to pick):** "Pick the epoch where valid loss first turns up while train loss keeps dropping; keep the checkpoint just before the upturn."
- **L2 vs L1 (why/when):** "L2 shrinks weights smoothly (Gaussian prior, stabilizes optimization); L1 promotes sparsity/feature selection (Laplace prior)."
- **Dropout (what it does):** "Train = randomly drop units (approx bagging of many thinned nets); test = scale activations, dropout off."
- **BN (train vs eval):** "Train uses batch mean/var; eval uses running stats; BN stabilizes internal covariate shift and allows larger learning rates."
- **Label smoothing (effect):** "Replace hard 1 with $1 - \alpha$ and others with α/K; reduces over-confidence, improves calibration and robustness to label noise."

2) CNN: shapes & spatial-relation caveat

- **Output shape (quick rule):** "Conv/Pool: $H_{out} = \lfloor (H + 2P - K)/S \rfloor + 1$ (same for W); SAME padding chooses P s.t. $H_{out} \approx H/S$."
- **Max vs Avg pool (intuition):** "Max keeps strongest part evidence (invariance, sparsity); Avg preserves overall presence but can dilute sharp features."
- **Receptive field growth:** "Deeper stacks grow the effective receptive field; small kernels × depth ≈ large contextual field."
- **Why 'wrong face' still fires:** "CNN fires on local parts (eyes/nose/mouth) without an explicit constraint on their **relative layout**; mis-arranged parts can still trigger a 'face' score."
- **Mitigations:** "Add relational/attention modules, positional encodings, or global structure losses; use data with hard negatives (wrong layouts)."

3) Adversarial examples & adversarial training

- **Targeted vs Untargeted:** "Untargeted pushes away from the true label (↑ loss for y); targeted pulls toward a chosen target t (↓ loss for t)."
- **FGSM vs PGD:** "FGSM = one big step; PGD = many small steps with projection—stronger, approximates inner maximization."
- **ϵ-ball meaning:** "ϵ bounds perceptual change under a chosen norm (often ℓ_∞); set by threat model not by accuracy alone."
- **Adversarial training vs augmentation:** "Both add samples; augmentation is label-preserving transforms, AT explicitly optimizes **worst-case** gradient-guided perturbations."
- **Common pitfall (masking):** "Gradient masking gives false security (non-useful gradients); always test with strong attacks (PGD, AutoAttack)."
- **Practical tip:** "Mix clean + adversarial in each batch; tune ϵ and steps; watch out for robust-overfitting—use early stopping on robust metrics."

4) RNN / LSTM / GRU

- **Why gradients vanish/explode:** "BPTT multiplies Jacobians across time; spectral radius <1 ⇒ decay (vanish), >1 ⇒ blow-up (explode)."
- **LSTM advantage:** "Cell state c_t and gates (input/forget/output) create a near-identity gradient path, capturing long-range dependencies."
- **GRU vs LSTM (choice):** "GRU has fewer gates (update/reset), merges states, trains faster on small/real time tasks; LSTM is steadier on very long contexts."
- **BPTT/truncation:** "Use truncated BPTT for long sequences to limit memory and stabilize training; clip gradients to curb explosions."
- **Output patterns:** "Sequence classification: use h_T (or pooled); sequence labeling: predict per step via $\hat{y}_t = \mathrm{softmax}(h_tV + c)$."

5) Word2Vec (Skip-gram / CBOW / Negative Sampling / Subsampling)

- **Goal in one line:** "Learn embeddings where words sharing contexts have nearby vectors (distributional hypothesis)."
- **SG vs CBOW:** "Skip-gram predicts context from center (better for rare words); CBOW predicts center from averaged context (faster, smoother)."
- **Negative sampling logic:** "Replace expensive softmax over V with K binary discriminations (real context vs noise), dropping cost to $O(K)$."
- **Why the 0.75 power:** "Softer than raw frequency—prevents head words from dominating yet keeps them represented."
- **Subsampling frequent words:** "Drop very frequent tokens with $1 - \sqrt{t/f(w)}$ to reduce noise and speed training."
- **Using embeddings:** "Similarity via cosine; average word vectors for sentence/doc baselines; analogies arise from linear regularities in embedding space."

6) Seq2Seq & Attention (incl. Q21 patterns)

- **BOS/EOS roles:** "BOS provides the first token/state to start decoding; EOS signals stop and enables variable-length outputs."
- **MLE with chain rule:** "Maximize $\sum_t \log P(y_t|y_{<t}, x)$; teacher forcing feeds gold $y_{<t}$ to stabilize training."
- **Fixed-context bottleneck:** "A single vector $c = h_{T_x}$ must summarize everything—hard for long inputs → information bottleneck."
- **Global attention (why/what):** "Compute $a_t = \mathrm{softmax}(\mathrm{score}(q_t, h_s))$, set $c_t = \sum_s a_t(s)h_s$; c_t adapts per step and aligns to relevant source words."
- **Score choices:** "dot (fast), general (learned matrix), concat (MLP); choose by cost vs flexibility."
- **Decoding tip:** "Use beam search to trade off exploration and probability; length penalties help avoid short outputs."

7) Transformer / ViT (attention, PE, shapes, complexity)

- **Why scale by $1/\sqrt{d_k}$:** "Keeps pre-softmax scores in a reasonable range so softmax doesn't saturate; gradients stay healthy."
- **Why multi-head helps:** "Different heads attend to different relations/scales/subspaces in parallel, improving expressivity and stability."
- **Positional information:** "Self-attention is permutation-invariant; add sinusoidal or learnable positional encodings to inject order."
- **Masking patterns:** "Causal mask for autoregressive decoding; cross-attention queries from decoder, keys/values from encoder."
- **ViT patch trade-off:** "Token count $N = \frac{HW}{P^2}$: larger P reduces cost but loses fine details; smaller P preserves detail but raises $O(N^2)$ attention cost."
- **[CLS] token purpose:** "A learnable global aggregator; after attention, its vector summarizes the image and feeds the classifier."
- **Practical tip:** "Use LayerNorm + residuals (often pre-LN) for stable deep training; FFN width ~4× hidden; watch memory at large N."

8) GAN (objective, updates, equilibrium, DCGAN, eval, issues)

- **Concept in one hit:** "G maps noise to fakes that fool D; D outputs real/fake probability—zero-sum game."
- **Training updates:** "Step-D: maximize $\log D(x) + \log(1 - D(G(z)))$; Step-G: minimize $\log(1 - D(G(z)))$ or maximize $\log D(G(z))$ (non-saturating for stronger gradients)."
- **Equilibrium fact:** "Optimal $D^*(x) = \frac{p_d}{p_d + p_g}$; at Nash equilibrium $p_g = p_d$ so $D = 0.5$ everywhere (indistinguishable)."
- **DCGAN recipe:** "G: ConvTranspose + BN + ReLU (Tanh at output); D: Strided Conv + LeakyReLU (Sigmoid at output)."
- **Evaluation:** "Inception Score: high per-image confidence + high dataset diversity; prefer FID for realism/diversity vs real data distribution."
- **Failure + fixes:** "Mode collapse & instability; mitigate with non-saturating G-loss, spectral norm/grad penalty, feature matching, minibatch discrimination, TTUR, balanced updates."
- **Small-data tip:** "Use strong aug for D, equalize updates, monitor D not to over-power G (keep D neither too weak nor too strong)."